PYTHON · HANDS-ON WORKSHOP

Python Fundamentals: From Variables to Automation

Read it. Run it. Change it.

Four hours · Four modules · One working script



SESSION MAP

Four hours, four building blocks

Module 1 · About 50 min

Variables, types, operators

Module 2 · About 50 min

Conditions and loops

Module 3 · About 60 min

Lists, tuples, sets, dictionaries

Module 4 · About 60 min

Functions and file automation

Plus two short breaks.

LEARNING RHYTHM

Every topic runs the same way

1. Concept: the idea in a few minutes
2. Live demo: code it together
3. Exercise: you try it, then we review

```
prediction = "What will this print?"
print(prediction)
```

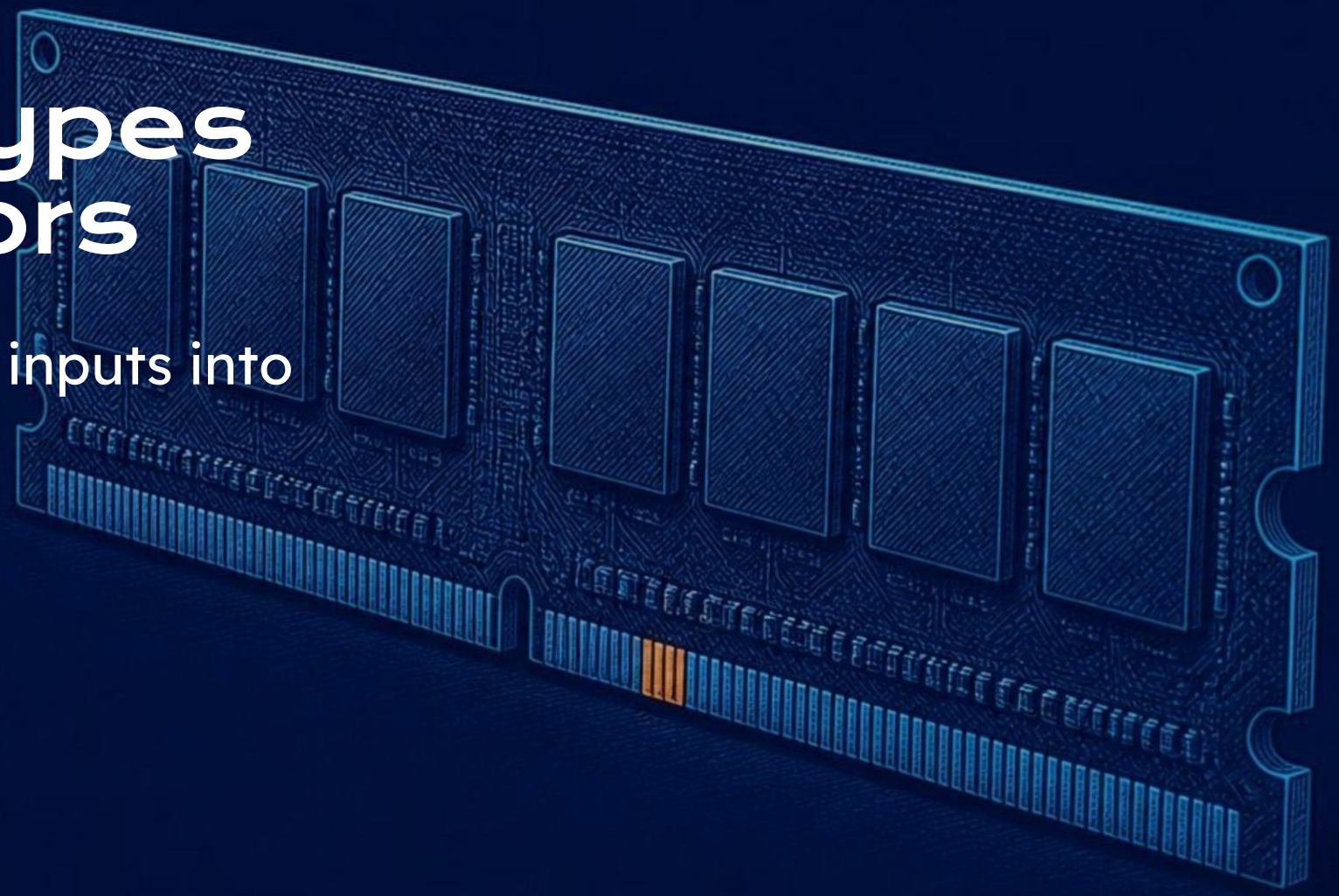
Predict the result before running each example.

MODULE 1

Variables, types and operators

Give values names. Turn inputs into results.

Build a unit converter.



MODULE 1 · CONCEPT + LIVE DEMO

A variable is a name pointing to a value

`=` creates or rebinds a name.

Use letters, digits and underscores, but never start with a digit. Avoid keywords; prefer `snake_case`.

The value carries the type, not the name.

```
item_count = 3
item_count += 1
print(item_count)  # 4
status = "ready"
status = True
```

Try it: rebind status to a number. What changes?

MODULE 1 · VALUES + STRINGS

Six core types cover most everyday code

```
count = 4
price = 2.50
name = "Python"
ready = True
result = None
items = [1, 2]
```

int · float · str · bool · NoneType · list

Strings are sequences

```
name[0]
name[1:4]
f"Hi, {name}"
```

Indexing starts at zero. A slice excludes its stop index.

Outputs: "P" · "yth" · "Hi, Python"

Try it: predict name[-1] and name[:2]. None represents an absent value.

MODULE 1 · CONVERSION

Convert and check types explicitly

```
int("12")  # 12
float("2.5")  # 2.5
str(12)  # "12"
bool("")  # False
bool("False")  # True
type(12)  # int
isinstance(2.5, float)  # True
raw = input("Count: ")
count = int(raw)
# "twelve" raises ValueError
```

Input arrives as text. Convert at the boundary; validate before using it.

Try it: compare bool("0") with bool(0).

MODULE 1 · EXPRESSIONS

Operators combine values into results

Arithmetic `+ - * / // % **` · `7 / 2, 7 // 2, 7 % 2` → `(3.5, 3, 1)`; `2 ** 3` → `8`.

Comparison `== != < > <= >=` · `3 < 5 and 5 != 0` → `True`.

Logical & membership `and, or, not; in` · `"py" in "python"` → `True`; `not False or False` → `True`.

Precedence: multiplication before addition; comparisons before `not`, then `and`, then `or`. Parentheses make intent visible.

Try it: predict 2 + 3 * 4, then (2 + 3) * 4.

MODULE 1 · PRACTICE LAB

Demo and exercise: unit converter

```
c = float(input("C: "))
f = c * 9 / 5 + 32
print(f"{f:.1f} °F")
```

Your turn

Add Fahrenheit-to-Celsius and kilometers-to-miles conversion.

Use 1 mile ≈ 1.609344 km.

Review

Test zero, a decimal and bad input. Round the display, not intermediate values.

MODULE 2

Conditions and loops

Choose a path. Repeat useful work.

Build a guessing game.



MODULE 2 · DECISION FLOW

if, elif, else choose one path

```
if score >= 80:
    print("Pass")
elif score >= 50:
    print("Retry")
else:
    print("Review")
```

score ≥ 80?

True → Pass

False ↓

score ≥ 50?

True → Retry

False ↓

Review

Indentation defines each body. The first true condition wins. Try score = 65.

MODULE 2 · READABLE DECISIONS

Build richer conditions without nesting

```
if ready and not paused:
    print("Run")

label = "yes" if ready
else "no"
```

`or` accepts either test; `and` needs both.

```
def first_item(items):
    if not items:
        return None
    return items[0]
```

A guard clause exits early, keeping the main path flat.

Falsy examples: False, None, 0, "", []. Try the function with [] and ["a"].

MODULE 2 · ITERATION

for loops repeat over any sequence

```
for letter in "cat":
    print(letter)
```

```
for i, name in enumerate(
    ["Ada", "Lin"], start=1
):
    print(i, name)
```

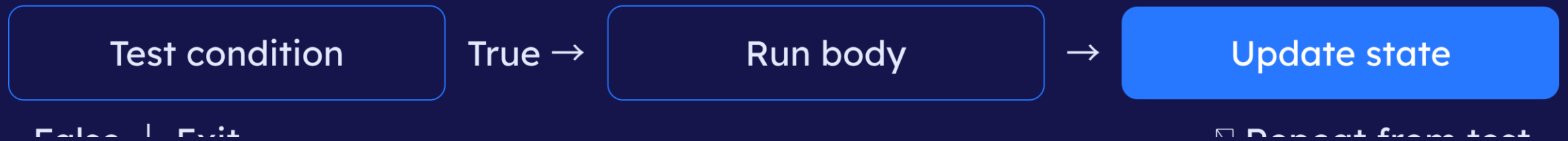
```
total = 0
for n in range(1, 5):
    total += n
print(total)  # 10
```

range() excludes its stop value.

Try it: accumulate only the even numbers from 1 through 10.

MODULE 2 · LOOP FLOW

while loops run until a condition changes




```
attempts = 3
while attempts > 0:
    attempts -= 1
    print(attempts)
```

`break` exits; `continue` skips to the next test. Update state before a skip, or the loop may never finish.

Try it: trace the condition before each iteration.

MODULE 2 · PRACTICE LAB

Demo and exercise: guess the number

LIVE DEMO · FIZZBUZZ

```
for n in range(1, 16):
    word = "Fizz"*(n%3==0) +
"Buzz"*(n%5==0)
    print(word or n)
```

Your turn

Pick a secret number. Allow five guesses, give higher/lower hints, and exit on a match.

Review

Why choose for or while? What ends the game? Does invalid input consume an attempt?

MODULE 3

Data structures

Choose how your program holds its data.

Build an inventory tracker.



MODULE 3 · LISTS

Lists hold ordered, changeable items

```
items = ["pen", "book"]
items.append("bag")
items.insert(0, "cup")
items.remove("pen")
last = items.pop()
print(items[:2])
```

```
alias = items
copy = items.copy()
```

Assignment shares a reference; a shallow copy creates a new outer list. `items.sort()` sorts in place; `sorted(items)` returns a new list. Try it: append through alias. Does copy change? What about nested lists?

MODULE 3 · TUPLES

Tuples are fixed records

Immutable once created.

Unpacking: `x, y = point`

Use tuples for coordinates, grouped return values and fixed records.

A tuple cannot replace its items. It works as a dictionary key only when all its contents are hashable.

Try it: unpack a three-item tuple into three names.

```
point = (3, 7)
x, y = point
single = (3,)
locations = {point:
"warehouse"}
point[0] = 9  # TypeError
```

The comma creates a one-item tuple.

MODULE 3 · SETS

Sets store unique items fast

```
a = {"pen", "book", "pen"}; b = {"book", "bag"}
a | b; a & b; a - b
"pen" in a  # True
empty = set()  # not {}
```

Automatic de-duplication; no positional indexing or guaranteed iteration order. Membership is typically fast: average constant time. Union (|), intersection (&), difference (-).

Try it: find categories shared by two inventories.

MODULE 3 · DICTIONARIES

Dictionaries map keys to values

```
stock = {"pen": 12}
stock["book"] = 4
stock["pen"] += 3
stock.get("bag", 0)  # 0
del stock["book"]
for item, qty in stock.items():
    print(item, qty)
```

```
inventory = {"pen": {"qty": 12,
"category": "office"}}
inventory["pen"]["qty"]
```

`keys()`, `values()` and `items()` expose views.

Try it: add a second nested record and read its category.

MODULE 3 · CHOOSING A COLLECTION

Pick the structure that fits the job

Type	Ordered	Mutable	Duplicates	Lookup speed*	Typical use
list	Yes	Yes	Allowed	Linear search	Task queue
tuple	Yes	No	Allowed	Linear search	Coordinates
set	No	Yes	No	Average constant	Unique tags
dict	Insertion	Yes	Unique keys	Average constant	Item quantities

*Membership search for list/tuple/set; key lookup for dict. List and tuple indexing is constant time. Dictionary values may repeat.

Try it: choose a structure for an inventory keyed by item name.

MODULE 3 · COMPREHENSIONS

Comprehensions build collections in one line

```
nums = [1, 2, 3, 4]
squares = [n*n for n in nums if n%2 == 0]  # [4, 16]
by_name = {name: len(name) for name in ["pen", "book"]}
categories = {item["category"] for item in
inventory.values()}
```

Read as “transform each item, keeping those that pass the filter.”

Use a plain loop when branching, side effects or nested logic need explanation.

Try it: build a dictionary containing only inventory items with quantity below 5.

MODULE 3 · PRACTICE LAB

Demo and exercise: inventory tracker

```
inventory = {"pen": {"qty": 12, "category": "office"},
"mug": {"qty": 2, "category": "kitchen"}}
inventory["pen"]["qty"] += 5
for name, item in inventory.items():
    if item["qty"] < 5: print(name, item["qty"])
```

Your turn

Add stock, find low items and list unique categories.

Review

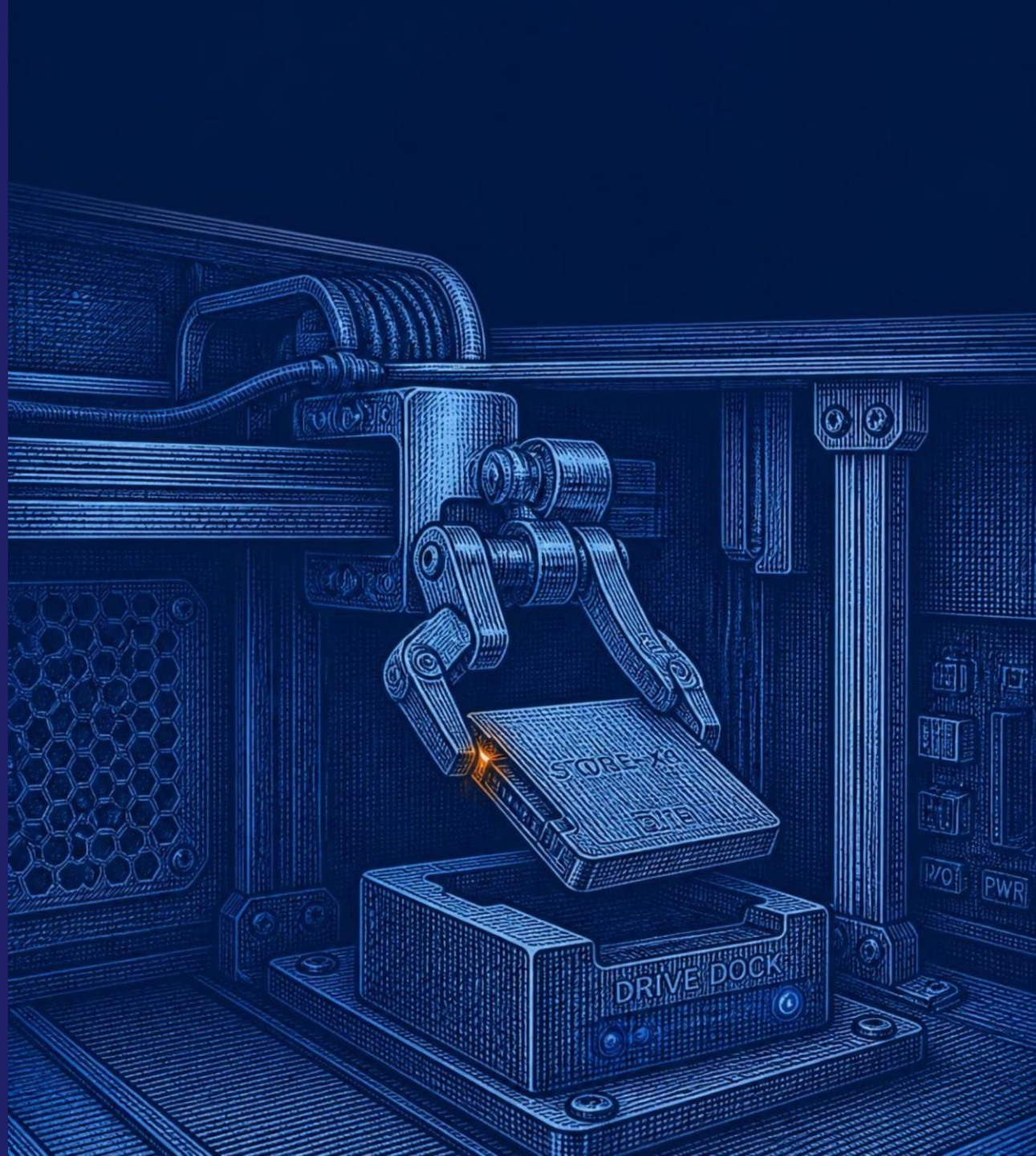
Why a dictionary for records? Why a set for categories? How will you handle a new item?

MODULE 4

Functions and automation

Package the logic. Let a script repeat it.

Build a file organizer.



MODULE 4 · REUSABLE LOGIC

Functions package reusable logic

```
def celsius_to_fahrenheit(celsius):
    """Convert Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32
print(celsius_to_fahrenheit(20))  # 68.0
print(celsius_to_fahrenheit(0))   # 32.0
```

`def` introduces a name, parameters and an indented body. A docstring explains intent. DRY means write once, call anywhere.

Try it: extract the distance conversion from Module 1 into a function.

MODULE 4 · FUNCTION CONTRACTS

Parameters in, return values out

`def total(price, qty=1): return price * qty`

`total(5, 3)` — positional

`total(qty=3, price=5)`

`total(5)` — default

```
def bounds(values): return min(values), max(values)

low, high = bounds([3, 8, 2])
```

`return` hands a value to the caller. `print` displays text; it returns None.

Try it: return a converted value first, then format it at the call site.

MODULE 4 · ORGANIZATION

Scope and modules keep code organized

Names assigned inside a function are local. Module-level names are global to that module.

Pass data explicitly instead of changing globals.

Import standard-library tools rather than rebuilding them.

Try it: import this file. The guard prevents main() from running on import.

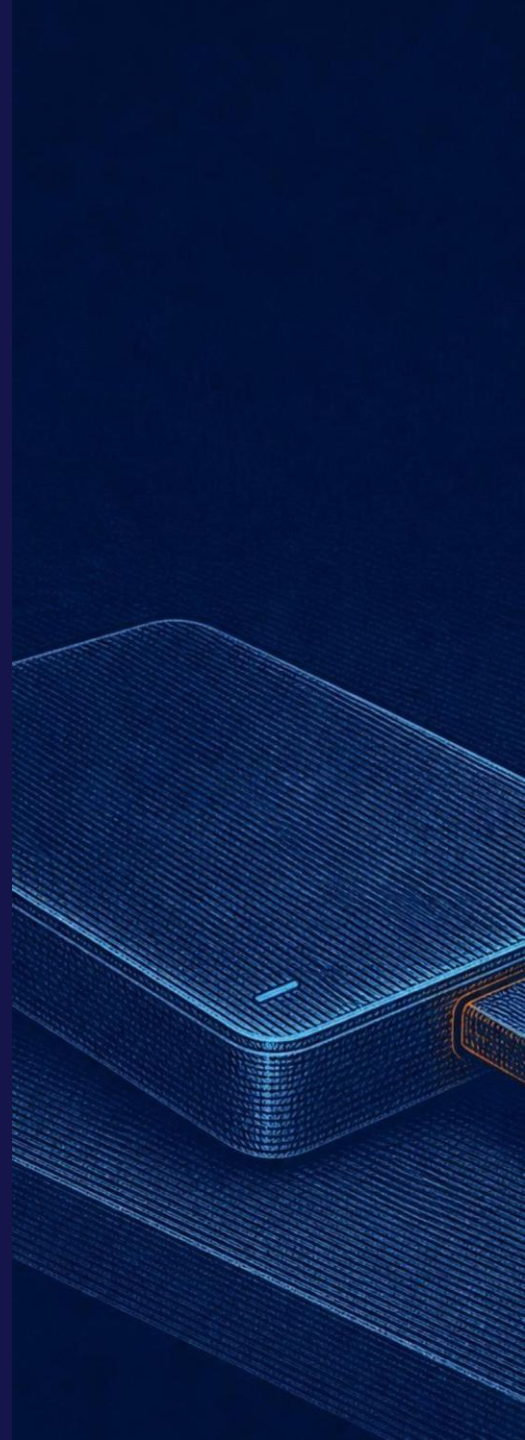
```
import random
def main():
    secret = random.randint(1,
10)
    print(secret)
if __name__ == "__main__":
main()
```

MODULE 4 · FILESYSTEM TOOLS

Scripts can work with files and folders

```
from pathlib import Path
print(list(Path("sandbox").iterdir()))
try: print(open("notes.txt").read())
except OSError: print("File error")
```

pathlib: list, read, and move files (Path.read_text() reads; write_text() writes; rename() moves). With open(), use "w" to write. Handle errors with try/except. Test on copies.



MODULE 4 · CAPSTONE

Capstone: a file organizer script



Scan a folder. Group by extension. Create subfolders and move files.

Start with a dry run. Use a sandbox; skip collisions and report errors.

```
from pathlib import Path
def organize(folder, dry_run=True):
    groups = {".jpg": "images"}
    for p in Path(folder).iterdir():
        if p.is_file():
            ext = p.suffix.lower()
            group = groups.get(ext, ext[1:] or
"other")
            dest = p.parent / group / p.name
            print(p, "->", dest)
            if not dry_run and not dest.exists():
                dest.parent.mkdir(exist_ok=True)
                p.rename(dest)
```

PRACTICE BEYOND THE WORKSHOP

What you can now build, and where to go next

Variables model values; conditions and loops control work; collections organize data; functions enable reusable automation.

Choose one useful script

Rename tools, log cleaners or CSV reports.

Next topics: file formats, modules and testing.

